

Abylay Utemis Rustem Gazymbekov

Task A

Target website

Site: SPIRITO — Barbershop & MarketBar

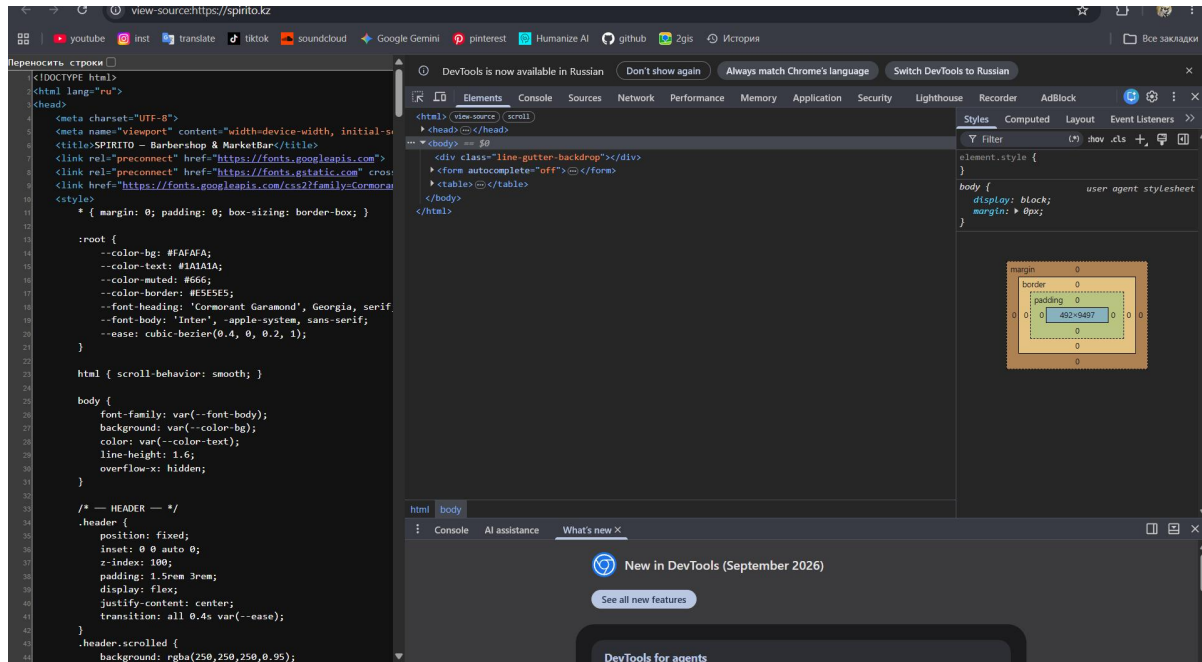
URL: <https://spirito.kz>

Reason for choosing this site: SPIRITO is a real barbershop business located in Astana. It matches our own project theme LOGOVO Barbershop closely enough to make a meaningful comparison of structural choices, while being an independent, unrelated business.

Technical details from <head>

Element	Value found in source
Doctype	<!DOCTYPE html>
lang attribute	lang="ru"
Charset	<meta charset="UTF-8">
Viewport	<meta name="viewport" content="width=device-width, initial-scale=1.0">
Title	SPIRITO — Barbershop & MarketBar
Additional meta tags	None found. The <head> contains only charset and viewport meta tags — no meta description, no Open Graph tags, no author tag.

Source code screenshot:



Semantic structure vs. div usage

What is done correctly:

Top-level skeleton uses real semantic tags: one `<header>`, three `<section>` elements, one `<footer>`.

Where div is used instead of a semantic tag:

perk-title / perk-text: a card's mini-heading and paragraph are both plain `<div>`s instead of `<h3> + <p>`.

location-name / location-address / location-phone: branch name, real address and phone number are three separate `<div>`s. The `<address>` element exists exactly for this purpose and is not used.

perks-list and mb-perks: four repeating, identical service-perk items are marked up as a set of `<div>`s rather than a `<ul><li>` list.

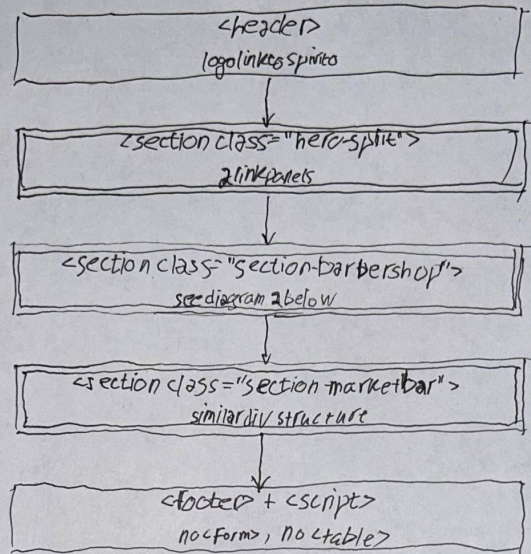
location-card: a self-contained, repeatable content block (one branch's info) is a `<div>` rather than an `<article>`.

Three structural mistakes and how we avoided them

#	Mistake found	Where in the source	How we avoided it in our own project
1	No <code><h1></code> anywhere on the page — headings jump straight to <code><h2></code> .	Every <code><h2></code> in the <code>.hero-split</code> and <code>.section</code> blocks (e.g. "Barbershop", "MarketBar", "Больше, чем стрижка")	Every one of our six pages has exactly one <code><h1></code> inside <code><header></code> — e.g. <code>index.html</code> : <code><h1>LOGOVO Barbershop</h1></code> .
2	All images are CSS <code>background-image</code> only — no <code></code> tag exists anywhere, so none can have an <code>alt</code> attribute.	<code>.panel-barbershop::before</code> / <code>.panel-marketbar::before</code> <code>background-image</code> rules	<code>gallery.html</code> uses real <code></code> tags for every photo, with a descriptive <code>alt</code> text for each (e.g. <code>facade</code> , <code>tools</code> , <code>awards</code>).
3	Repeating, same-type content blocks (services, branches) are marked up as generic <code><div></code> s instead of a list.	<code>.perks-list</code> , <code>.mb-perks</code> , <code>.location-card</code>	<code>services.html</code> uses a real nested <code></code> for our list of services and categories, not <code>div</code> wrappers.

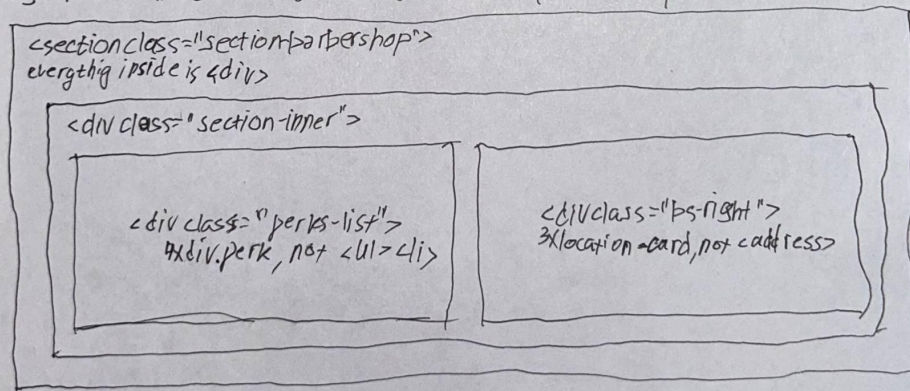
Hand-drawn structure

Diagram 1 — top-level structure of the SPIRITO page



Double border = contains div used instead of a semantic tag

Diagram 2 — Inside <section class="section-barbershop">



Rustem Gazymbekov, Abylay Utemis

Date: 13.09.2026

Task B — Written part

1. What a web page is made of and how the browser shows it

A web page is a text file with the extension .html. Inside it there is our content (words, prices, names) and tags around the content. A tag tells the browser what the content is: h1 is the main heading, p is a paragraph, table is a table. Images are separate files in our images/ folder, and the `img src="..."` tag only points to them.

When we open index.html, the browser reads the file from top to bottom. First it reads head: charset says how to read the letters, viewport says how to fit the page on a phone, and title goes to the browser tab. Nothing from head is shown on the page.

Then the browser reads body and builds a tree of elements: body contains header, nav, main and footer; main contains sections with headings and paragraphs. This tree is called the DOM. From the tree the browser draws the page. We have no CSS, so the browser uses its default style: h1 is big and bold, ul has bullets, a is blue and underlined. When the browser meets img, it loads the picture file and puts it in that place.

2. Three places where we chose a semantic tag instead of div

a) nav with ul on every page — index.html, line 19. We could write `div class="menu"` with six links inside. But nav tells the browser and a screen reader "this is the navigation of the site", and ul says "these six links are one group". A div says nothing about its content. The site we studied in Task A (Spirito) has no nav at all, so its menu does not exist as a structure.

b) aside for the block "In Numbers" — about.html, line 43. The numbers (2000 reviews, rating 5.0, 20 barbers) are extra information next to the main text, not part of the story. aside means exactly this: "related, but on the side". With a div the browser would see just a box. Inside the aside we did use one div for the badge with the number 5.0 (about.html, line 52), because it is not a list, not a paragraph and not a section. We wrote a comment there to explain it.

c) table for the price list — services.html, line 72. Every service has three prices (lead, top, senior barber). With divs the connection between a service and its price would exist only visually. In a table with `thead` and `tbody` (lines 76–79 and 84–138), the browser knows that "4000 tg" belongs to the row "Men's haircut" and the column "Top barber". A screen reader can read it as "Men's haircut, Top barber, 4000 tg".

3. What happens when a visitor presses the Submit button today

Our form is in booking.html, line 80: `form method="post" action="#"`.

When a visitor presses "Book", the browser first checks the form itself, without any server. Fields with the required attribute (name, email, phone, date) must not be empty; the `type="email"` field must contain @; the `type="number"` field (line 113) must be between `min="1"` and `max="5"`. If something is wrong, the browser shows a small message near the field and does not send anything.

If all checks pass, the browser collects the values of all fields by their name attributes (client-name, client-email, barber-level, service-select and so on) and makes a POST request to the address in action. Our action is "#", which means "this same page". There is no server behind it, so nobody receives the data. The browser just reloads booking.html and the form becomes empty again. That is why we wrote the comment inside the form (line 81): "there is no server yet, so the form does not send anything".